```
{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "# Lecture 10 (Part a)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Public, Protected, Private\n",
    "\n",
    "In most objected oriented languages, data encapsulation is achieved by enabling classes to declair protected and private data members in addition to the public ones. Quick recap:\n",
    "\n",
    "* Public members are accessible to everyone\n",
    "* Private members are only accessible to the class itself\n",
    "* Protected members are accessiblve to the all instances of the same class (including child classes) \n",
    "\n",
    "Python's implementation of data encapsulation is not strictly enforced by the langauge and is mostly a convention. Data members starting with one underscore (`_`) are protected. Data members starting with two underscores (`__`) are private.\n",
    "\n",
    "Consider the following example:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 1,
   "metadata": {},
   "outputs": [],
   "source": [
    "class parent:\n",
    "    def __init__(self):\n",
    "        self.public=\"I'm Public\"\n",
    "        self._protected=\"I'm Protected\"\n",
    "        self.__private=\"I'm Private\"\n",
    "        \n",
    "    def set_private_parent(self,v):\n",
    "        self.__private=v\n",
    "        \n",
    "    def get_private_parent(self):\n",
    "        return self.__private\n",
    "        \n",
    "class child(parent):\n",
    "    def __init__(self, init_parent=True):\n",
    "        if init_parent:\n",
    "            super(child,self).__init__()\n",
```

```
    "    \n",
    "    def set_public(self,v):\n",
    "        self.public=v\n",
    "        \n",
    "    def set_protected(self,v):\n",
    "        self._protected=v\n",
    "\n",
    "    def set_private(self,v):\n",
    "        self.__private=v\n",
    "        \n",
    "    def get_public(self):\n",
    "        return self.public\n",
    "        \n",
    "    def get_protected(self):\n",
    "        return self._protected\n",
    "\n",
    "    def get_private(self):\n",
    "        return self.__private\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Lets see what we can do with the private attributes:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 2,
   "metadata": {},
   "outputs": [
    {
     "ename": "AttributeError",
     "evalue": "'parent' object has no attribute '__private'",
     "output_type": "error",
     "traceback": [
      "\u001b[0;31m---------------------------------------------------------
---------------------\u001b[0m",
      "\u001b[0;31mAttributeError\u001b[0m
Traceback (most recent call last)",
      "Cell \u001b[0;32mIn[2], line 3\u001b[0m\n\u001b[1;32m
1\u001b[0m \u001b[38;5;66;03m# Create an instance of
parent:\u001b[39;00m\n\u001b[1;32m      2\u001b[0m my_parent
\u001b[38;5;241m=\u001b[39m parent()\n\u001b[0;32m----> 3\u001b[0m
\u001b[38;5;28mprint\u001b[39m(\u001b[43mmy_parent\u001b[49m\u001b[38;5;2
41;43m.\u001b[39;49m\u001b[43m__private\u001b[49m)\n",
      "\u001b[0;31mAttributeError\u001b[0m: 'parent' object has no
attribute '__private'"
     ]
    }
   ],
   "source": [
    "# Create an instance of parent:\n",
```

```
    "my_parent = parent()\n",
    "print(my_parent.__private)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "We can't access the private attributes from outside of the class. It
should be the same with the protected:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 3,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "I'm Protected\n"
     ]
    }
   ],
   "source": [
    "print(my_parent._protected)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 4,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "abc\n"
     ]
    }
   ],
   "source": [
    "my_parent._protected = 'abc'\n",
    "print(my_parent._protected)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "But its not! We can access protected data!\n",
    "\n",
    "Finally, lets check out public attributes:"
```

```
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 5,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "I'm Public\n",
      "5\n"
     ]
    }
   ],
   "source": [
    "print(my_parent.public)\n",
    "my_parent.public = 5\n",
    "print(my_parent.public)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Note that we can set attributes of the class instead of the
instance. "
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 6,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "22\n",
      "5\n"
     ]
    }
   ],
   "source": [
    "parent.public = 22\n",
    "print(parent.public)\n",
    "print(my_parent.public)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
```

```
    "But it doesn't make sense to do so. \n",
    "\n",
    "The proper way to access/change a private attribute is using the
accessors set setters provided by the class:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 7,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "42"
      ]
     },
     "execution_count": 7,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "my_parent.get_private_parent()\n",
    "my_parent.set_private_parent(42)\n",
    "my_parent.get_private_parent()"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Note that because we declared the data members in the parent
constructor, we have to make sure that the child calls the parent
constructor."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 8,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "22"
      ]
     },
     "execution_count": 8,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
```

```
    "child_instance = child(init_parent=False)\n",
    "child_instance.public"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "We set this data member earlier, that's why its value is 22. It
should have been \"I'm public\", but we didn't call the constructor of
the parent class. \n",
    "\n",
    "If everything is done correctly:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 9,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "\"I'm Public\""
      ]
     },
     "execution_count": 9,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "child_instance = child()\n",
    "child_instance.public"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Even though we wrote accessors (setters and getters), we can
directly access and set the data:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 10,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "I'm Public\n",
```

```
    "Changed Public\n"
   ]
  }
 ],
 "source": [
  "child_instance = child()\n",
  "print(child_instance.public)\n",
  "child_instance.public=\"Changed Public\"\n",
  "print(child_instance.public)"
 ]
},
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "Of couse the accessors also work:"
 ]
},
{
 "cell_type": "code",
 "execution_count": 11,
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "I'm Public\n",
    "Changed Public\n"
   ]
  }
 ],
 "source": [
  "child_instance = child()\n",
  "print(child_instance.get_public())\n",
  "child_instance.set_public(\"Changed Public\")\n",
  "print(child_instance.get_public())"
 ]
},
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "How about the protected?"
 ]
},
{
 "cell_type": "code",
 "execution_count": 12,
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
```

```
     "text": [
      "I'm Protected\n",
      "Changed Protected\n"
     ]
    }
   ],
   "source": [
    "print(child_instance._protected)\n",
    "child_instance._protected=\"Changed Protected\"\n",
    "print(child_instance._protected)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 13,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "I'm Protected\n",
      "Changed Protected\n"
     ]
    }
   ],
   "source": [
    "child_instance = child()\n",
    "print(child_instance.get_protected())\n",
    "child_instance.set_protected(\"Changed Protected\")\n",
    "print(child_instance.get_protected())"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "So there isn't any difference between public and protected in
python. We can just adopt the convention that data members starting with
a single underscore will be only accessed via accessors. \n",
    "\n",
    "How about private?"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 14,
   "metadata": {},
   "outputs": [
    {
     "ename": "AttributeError",
     "evalue": "'child' object has no attribute '__private'",
     "output_type": "error",
     "traceback": [
```

```
      "\u001b[0;31m---------------------------------------------------------
---------------------\u001b[0m",
      "\u001b[0;31mAttributeError\u001b[0m
Traceback (most recent call last)",
      "Cell \u001b[0;32mIn[14], line 1\u001b[0m\n\u001b[0;32m----> 
1\u001b[0m
\u001b[38;5;28mprint\u001b[39m(\u001b[43mchild_instance\u001b[49m\u001b[3
8;5;241;43m.\u001b[39;49m\u001b[43m__private\u001b[49m)\n\u001b[1;32m
2\u001b[0m
child_instance\u001b[38;5;241m.\u001b[39m__private\u001b[38;5;241m=\u001b
[39m\u001b[38;5;124m\"\u001b[39m\u001b[38;5;124mChanged
Private\u001b[39m\u001b[38;5;124m\"\u001b[39m\n\u001b[1;32m
3\u001b[0m
\u001b[38;5;28mprint\u001b[39m(child_instance\u001b[38;5;241m.\u001b[39m_
_private)\n",
      "\u001b[0;31mAttributeError\u001b[0m: 'child' object has no
attribute '__private'"
     ]
    }
   ],
   "source": [
    "print(child_instance.__private)\n",
    "child_instance.__private=\"Changed Private\"\n",
    "print(child_instance.__private)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "It appears that we finally have some protection. A closer look shows
how its done:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 15,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "['__class__',\n",
       " '__delattr__',\n",
       " '__dict__',\n",
       " '__dir__',\n",
       " '__doc__',\n",
       " '__eq__',\n",
       " '__format__',\n",
       " '__ge__',\n",
       " '__getattribute__',\n",
       " '__gt__',\n",
       " '__hash__',\n",
       " '__init__',\n",
```

```
       " '__init_subclass__',\n",
       " '__le__',\n",
       " '__lt__',\n",
       " '__module__',\n",
       " '__ne__',\n",
       " '__new__',\n",
       " '__reduce__',\n",
       " '__reduce_ex__',\n",
       " '__repr__',\n",
       " '__setattr__',\n",
       " '__sizeof__',\n",
       " '__str__',\n",
       " '__subclasshook__',\n",
       " '__weakref__',\n",
       " '_parent__private',\n",
       " '_protected',\n",
       " 'get_private',\n",
       " 'get_private_parent',\n",
       " 'get_protected',\n",
       " 'get_public',\n",
       " 'public',\n",
       " 'set_private',\n",
       " 'set_private_parent',\n",
       " 'set_protected',\n",
       " 'set_public']"
      ]
     },
     "execution_count": 15,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "dir(child_instance)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Note that instead of a data member `__private` we have a data member `_parent__private`. All python does is to replace anything that has the pattern `<class_name>.__<data_name>` to `<class_name>._<class_name>__<data_name>`. So in fact, we can still change this data member and there is no real protection:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 16,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
```

```
     "output_type": "stream",
     "text": [
      "I'm Private\n",
      "Changed Private\n"
     ]
    }
   ],
   "source": [
    "print(child_instance._parent__private)\n",
    "child_instance._parent__private=\"Changed Private\"\n",
    "print(child_instance._parent__private)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Just to make sure we understand, here what the parent class looks
like:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 17,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "['__class__',\n",
       " '__delattr__',\n",
       " '__dict__',\n",
       " '__dir__',\n",
       " '__doc__',\n",
       " '__eq__',\n",
       " '__format__',\n",
       " '__ge__',\n",
       " '__getattribute__',\n",
       " '__gt__',\n",
       " '__hash__',\n",
       " '__init__',\n",
       " '__init_subclass__',\n",
       " '__le__',\n",
       " '__lt__',\n",
       " '__module__',\n",
       " '__ne__',\n",
       " '__new__',\n",
       " '__reduce__',\n",
       " '__reduce_ex__',\n",
       " '__repr__',\n",
       " '__setattr__',\n",
       " '__sizeof__',\n",
       " '__str__',\n",
       " '__subclasshook__',\n",
```

```
       " '__weakref__',\n",
       " '_parent__private',\n",
       " '_protected',\n",
       " 'get_private_parent',\n",
       " 'public',\n",
       " 'set_private_parent']"
      ]
     },
     "execution_count": 17,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "parent_instance=parent()\n",
    "dir(parent_instance)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Note that a child cannot change a parent's private data:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 18,
   "metadata": {},
   "outputs": [
    {
     "ename": "AttributeError",
     "evalue": "'child' object has no attribute '_child__private'",
     "output_type": "error",
     "traceback": [
      "\u001b[0;31m---------------------------------------------------------
---------------------\u001b[0m",
      "\u001b[0;31mAttributeError\u001b[0m
Traceback (most recent call last)",
      "Cell \u001b[0;32mIn[18], line 2\u001b[0m\n\u001b[1;32m
1\u001b[0m child_instance \u001b[38;5;241m=\u001b[39m
child()\n\u001b[0;32m----> 2\u001b[0m
\u001b[38;5;28mprint\u001b[39m(\u001b[43mchild_instance\u001b[49m\u001b[3
8;5;241;43m.\u001b[39;49m\u001b[43mget_private\u001b[49m\u001b[43m(\u001b
[49m\u001b[43m)\u001b[49m)\n\u001b[1;32m      3\u001b[0m
child_instance\u001b[38;5;241m.\u001b[39mset_private(\u001b[38;5;124m\"\u
001b[39m\u001b[38;5;124mChanged
Private\u001b[39m\u001b[38;5;124m\"\u001b[39m)\n\u001b[1;32m
4\u001b[0m
\u001b[38;5;28mprint\u001b[39m(child_instance\u001b[38;5;241m.\u001b[39mg
et_private())\n",
      "Cell \u001b[0;32mIn[1], line 34\u001b[0m, in
\u001b[0;36mchild.get_private\u001b[0;34m(self)\u001b[0m\n\u001b[1;32m
33\u001b[0m \u001b[38;5;28;01mdef\u001b[39;00m
```

```
\u001b[38;5;21mget_private\u001b[39m(\u001b[38;5;28mself\u001b[39m):\n\u0
01b[0;32m---> 34\u001b[0m     \u001b[38;5;28;01mreturn\u001b[39;00m
\u001b[38;5;28;43mself\u001b[39;49m\u001b[38;5;241;43m.\u001b[39;49m\u001
b[43m__private\u001b[49m\n",
      "\u001b[0;31mAttributeError\u001b[0m: 'child' object has no
attribute '_child__private'"
     ]
    }
   ],
   "source": [
    "child_instance = child()\n",
    "print(child_instance.get_private())\n",
    "child_instance.set_private(\"Changed Private\")\n",
    "print(child_instance.get_private())"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 19,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "I'm Private\n",
      "Changed Private\n"
     ]
    }
   ],
   "source": [
    "child_instance = child()\n",
    "print(child_instance.get_private_parent())\n",
    "child_instance.set_private_parent(\"Changed Private\")\n",
    "print(child_instance.get_private_parent())"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Why don't I get an error in the following case? "
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 20,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "I'm Private\n",
```

```
    "I'm Private\n"
   ]
  }
 ],
 "source": [
  "child_instance = child()\n",
  "print(child_instance.get_private_parent())\n",
  "child_instance.set_private(\"Changed Private\")\n",
  "print(child_instance.get_private_parent())"
 ]
},
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "Note that the code doesn't work as intended... why? \n",
  "\n",
  "See if you can figure it out by looking at:"
 ]
},
{
 "cell_type": "code",
 "execution_count": 21,
 "metadata": {},
 "outputs": [
  {
   "data": {
    "text/plain": [
     "['__class__',\n",
     " '__delattr__',\n",
     " '__dict__',\n",
     " '__dir__',\n",
     " '__doc__',\n",
     " '__eq__',\n",
     " '__format__',\n",
     " '__ge__',\n",
     " '__getattribute__',\n",
     " '__gt__',\n",
     " '__hash__',\n",
     " '__init__',\n",
     " '__init_subclass__',\n",
     " '__le__',\n",
     " '__lt__',\n",
     " '__module__',\n",
     " '__ne__',\n",
     " '__new__',\n",
     " '__reduce__',\n",
     " '__reduce_ex__',\n",
     " '__repr__',\n",
     " '__setattr__',\n",
     " '__sizeof__',\n",
     " '__str__',\n",
     " '__subclasshook__',\n",
     " '__weakref__',\n",
```

```
      " '_child__private',\n",
      " '_parent__private',\n",
      " '_protected',\n",
      " 'get_private',\n",
      " 'get_private_parent',\n",
      " 'get_protected',\n",
      " 'get_public',\n",
      " 'public',\n",
      " 'set_private',\n",
      " 'set_private_parent',\n",
      " 'set_protected',\n",
      " 'set_public']"
     ]
    },
    "execution_count": 21,
    "metadata": {},
    "output_type": "execute_result"
   }
  ],
  "source": [
   "dir(child_instance)"
  ]
 },
 {
  "cell_type": "markdown",
  "metadata": {},
  "source": [
   "## Data Serialization\n",
   "\n",
   "Data serialization refers to the process of converting data (usually in memory) that may have complex structure (e.g. a tree), into a linear sequence that can be use to reconstitute the original data structure. Such a sequence can be stored in a file or transmitted over a network.\n",
   "\n",
   "For example consider the following \"simple\" data structure:"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 22,
  "metadata": {},
  "outputs": [],
  "source": [
   "# Simple Data Type\n",
   "\n",
   "data_dict = { \"A\": 1, \n",
   "              \"B\": \"Foo\"}"
  ]
 },
 {
  "cell_type": "markdown",
  "metadata": {},
  "source": [
```

```
    "### Python `repr`\n",
    "\n",
    "The python `repr` method of build-ins and classes you implement can be used as a means of serialization. Take any python built in and you can see it's string representation, which is essentially a string of python code that can evaluates to the object:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 23,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "\"{'A': 1, 'B': 'Foo'}\""
      ]
     },
     "execution_count": 23,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "repr(data_dict)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "This representation can be easily written to a file:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 24,
   "metadata": {},
   "outputs": [],
   "source": [
    "with open('file.py',\"w\") as f: \n",
    "    f.write(repr(data_dict))"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 25,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
```

```
      "{'A': 1, 'B': 'Foo'}"
     ]
    }
   ],
   "source": [
    "!cat file.py"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "And reconstituted by evaluating the contents of the file:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 26,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "{'A': 1, 'B': 'Foo'}"
      ]
     },
     "execution_count": 26,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "with open('file.py', 'r') as f: \n",
    "    data_dict_reloaded = eval(f.read())\n",
    "\n",
    "data_dict_reloaded"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Note that `eval` uses the python interpreter to execute python
expressions stored in strings:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 27,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
```

```
     "text": [
      "Hello World\n"
     ]
    }
   ],
   "source": [
    "eval(\"print('Hello World')\")"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 28,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "2"
      ]
     },
     "execution_count": 28,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "x=eval(\"1+1\")\n",
    "x"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### YAML\n",
    "\n",
    "There are other standard formats for storing simple data types. For
example YAML:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 29,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "'A: 1\\nB: Foo\\n'"
      ]
     },
     "execution_count": 29,
     "metadata": {},
     "output_type": "execute_result"
```

```
    }
   ],
   "source": [
    "import yaml\n",
    "yaml.dump(data_dict)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 30,
   "metadata": {},
   "outputs": [],
   "source": [
    "with open('file.yaml',\"w\") as f: \n",
    "    f.write(yaml.dump(data_dict))"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 31,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "A: 1\r\n",
      "B: Foo\r\n"
     ]
    }
   ],
   "source": [
    "!cat file.yaml"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 32,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "\u001b[31mLecture.10.a.ipynb\u001b[m\u001b[m M.pickle
file.json          file.yaml\r\n",
      "\u001b[31mLecture.10.b.ipynb\u001b[m\u001b[m M_list.pickle
file.pickle\r\n",
      "M.npy              \u001b[31mScores.csv\u001b[m\u001b[m
file.py\r\n"
     ]
    }
   ],
   "source": [
```

```
    "!ls "
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 33,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "{'A': 1, 'B': 'Foo'}"
      ]
     },
     "execution_count": 33,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "with open('file.yaml', 'r') as f: \n",
    "    data_dict_reloaded = yaml.safe_load(f.read())\n",
    "\n",
    "data_dict_reloaded"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### JSON\n",
    "\n",
    "[JSON](https://www.json.org/json-en.html) is commonly used to
transmit data on the web:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 34,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "'{\"A\": 1, \"B\": \"Foo\"}'"
      ]
     },
     "execution_count": 34,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "import json\n",
```

```
  "json.dumps(data_dict)"
 ]
},
{
 "cell_type": "code",
 "execution_count": 35,
 "metadata": {},
 "outputs": [],
 "source": [
  "with open('file.json',\"w\") as f: \n",
  "    json.dump(data_dict,f)"
 ]
},
{
 "cell_type": "code",
 "execution_count": 36,
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "{\"A\": 1, \"B\": \"Foo\"}"
   ]
  }
 ],
 "source": [
  "!cat file.json"
 ]
},
{
 "cell_type": "code",
 "execution_count": 37,
 "metadata": {},
 "outputs": [
  {
   "data": {
    "text/plain": [
     "{'A': 1, 'B': 'Foo'}"
    ]
   },
   "execution_count": 37,
   "metadata": {},
   "output_type": "execute_result"
  }
 ],
 "source": [
  "with open('file.json', 'r') as f: \n",
  "    data_dict_reloaded = json.load(f)\n",
  "\n",
  "data_dict_reloaded"
 ]
},
{
```

```
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### XML\n",
    "\n",
    "XML is another format commonly used for storing data. It allows a
bit more structure and there are python tools for creating XML
representations of data, but it's a bit more complicated than the example
above, so we'll skip it for now."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### pickle\n",
    "\n",
    "[pickle](https://docs.python.org/3/library/pickle.html) is python's
method of serialing objects. Some advantages are that it is a binary
format, so it is more compact, and that it can store full python objects,
not just simple built-ins. Lets look at the [pickle
documentation](https://docs.python.org/3/library/pickle.html) first.\n",
    "\n",
    "Here is an example:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 38,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [

"b'\\x80\\x02}q\\x00(X\\x01\\x00\\x00\\x00Aq\\x01K\\x01X\\x01\\x00\\x00\\
x00Bq\\x02X\\x03\\x00\\x00\\x00Fooq\\x03u.'"
      ]
     },
     "execution_count": 38,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "import pickle\n",
    "pickle.dumps(data_dict,protocol=2)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 39,
   "metadata": {},
   "outputs": [],
```

```
   "source": [
    "with open('file.pickle',\"wb\") as f: \n",
    "    pickle.dump(data_dict,f)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 40,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [

"ï¿½\u0004ï¿½\u0015\u0000\u0000\u0000\u0000\u0000\u0000\u0000}ï¿½(ï¿½\u00
01Aï¿½K\u0001ï¿½\u0001Bï¿½ï¿½\u0003Fooï¿½u."
     ]
    }
   ],
   "source": [
    "!cat file.pickle"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 41,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "{'A': 1, 'B': 'Foo'}"
      ]
     },
     "execution_count": 41,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "with open('file.pickle', 'rb') as f: \n",
    "    data_dict_reloaded = pickle.load(f)\n",
    "\n",
    "data_dict_reloaded"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Python classes\n",
    "\n",
    "Imagine you have data stored in a python object:"
```

```
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 42,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Value of A: 1\n",
      "Value of B: Foo\n"
     ]
    }
   ],
   "source": [
    "# Instance of a python class with data\n",
    "\n",
    "class data_class:\n",
    "    def __init__(self):\n",
    "        self._data = dict()\n",
    "    \n",
    "    def add(self,key,value):\n",
    "        self._data[key]=value\n",
    "        \n",
    "    def get(self,key):\n",
    "        return self._data[key]\n",
    "    \n",
    "    def __repr__(self):\n",
    "        return self._data.__repr__()\n",
    "\n",
    "data_class_instance = data_class()\n",
    "data_class_instance.add(\"A\",1)\n",
    "data_class_instance.add(\"B\",\"Foo\")\n",
    "\n",
    "print(\"Value of A:\", data_class_instance.get(\"A\"))\n",
    "print(\"Value of B:\", data_class_instance.get(\"B\"))"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Since we implemented `__repr__`, I should be able to store the data
using `repr`:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 43,
   "metadata": {},
   "outputs": [],
   "source": [
```

```
    "with open('file.py',\"w\") as f: \n",
    "    f.write(repr(data_class_instance))"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 44,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "{'A': 1, 'B': 'Foo'}"
     ]
    }
   ],
   "source": [
    "!cat file.py"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 45,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "total 47272\r\n",
      "-rwxr--r--@ 1 afarbin  staff    36258 Feb 17 13:40
\u001b[31mLecture.10.a.ipynb\u001b[m\u001b[m\r\n",
      "-rwxr-xr-x@ 1 afarbin  staff   134721 Feb 17 11:36
\u001b[31mLecture.10.b.ipynb\u001b[m\u001b[m\r\n",
      "-rw-r--r--@ 1 afarbin  staff  8000128 Feb  7 10:37 M.npy\r\n",
      "-rw-r--r--@ 1 afarbin  staff  8000163 Feb  7 10:37 M.pickle\r\n",
      "-rw-r--r--@ 1 afarbin  staff  8000163 Feb  7 10:37
M_list.pickle\r\n",
      "-rwxr--r--@ 1 afarbin  staff     2428 Feb  7 10:37
\u001b[31mScores.csv\u001b[m\u001b[m\r\n",
      "-rw-r--r--@ 1 afarbin  staff       20 Feb 17 13:32 file.json\r\n",
      "-rw-r--r--@ 1 afarbin  staff       32 Feb 17 13:38
file.pickle\r\n",
      "-rw-r--r--@ 1 afarbin  staff       20 Feb 17 13:41 file.py\r\n",
      "-rw-r--r--@ 1 afarbin  staff       12 Feb 17 13:30 file.yaml\r\n"
     ]
    }
   ],
   "source": [
    "!ls -l"
   ]
  },
  {
```

```
   "cell_type": "code",
   "execution_count": 46,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "{'A': 1, 'B': 'Foo'}"
      ]
     },
     "execution_count": 46,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "with open('file.py', 'r') as f: \n",
    "    data_class_instance_reloaded = eval(f.read())\n",
    "\n",
    "data_class_instance_reloaded"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "But what I get back is not the original object reconstituted, but a
dictionary holding the data:"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 47,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "dict"
      ]
     },
     "execution_count": 47,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "type(data_class_instance_reloaded)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 48,
   "metadata": {},
```

```
   "outputs": [
    {
     "ename": "AttributeError",
     "evalue": "'dict' object has no attribute 'add'",
     "output_type": "error",
     "traceback": [
      "\u001b[0;31m---------------------------------------------------------
---------------------\u001b[0m",
      "\u001b[0;31mAttributeError\u001b[0m
Traceback (most recent call last)",
      "Cell \u001b[0;32mIn[48], line 1\u001b[0m\n\u001b[0;32m---->
1\u001b[0m
\u001b[43mdata_class_instance_reloaded\u001b[49m\u001b[38;5;241;43m.\u001
b[39;49m\u001b[43madd\u001b[49m(\u001b[38;5;124m\"\u001b[39m\u001b[38;5;1
24mC\u001b[39m\u001b[38;5;124m\"\u001b[39m,\u001b[38;5;241m2\u001b[39m)\n
",
      "\u001b[0;31mAttributeError\u001b[0m: 'dict' object has no
attribute 'add'"
     ]
    }
   ],
   "source": [
    "data_class_instance_reloaded.add(\"C\",2)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 49,
   "metadata": {},
   "outputs": [
    {
     "data": {
      "text/plain": [
       "{'A': 1, 'B': 'Foo'}"
      ]
     },
     "execution_count": 49,
     "metadata": {},
     "output_type": "execute_result"
    }
   ],
   "source": [
    "data_class_instance_reloaded"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "We can modify the class to work."
   ]
  },
  {
   "cell_type": "code",
```

```
 "execution_count": 50,
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "Value of A: 1\n",
    "Value of B: Foo\n"
   ]
  }
 ],
 "source": [
  "# Instance of a python class with data\n",
  "\n",
  "class data_class:\n",
  "    def __init__(self,d=None):\n",
  "        if d:\n",
  "            self._data=d\n",
  "        else:\n",
  "            self._data = dict()\n",
  "    \n",
  "    def add(self,key,value):\n",
  "        self._data[key]=value\n",
  "        \n",
  "    def get(self,key):\n",
  "        return self._data[key]\n",
  "    \n",
  "    def __repr__(self):\n",
  "        return \"data_class(\"+self._data.__repr__()+\")\"\n",
  "\n",
  "data_class_instance = data_class()\n",
  "data_class_instance.add(\"A\",1)\n",
  "data_class_instance.add(\"B\",\"Foo\")\n",
  "\n",
  "print(\"Value of A:\", data_class_instance.get(\"A\"))\n",
  "print(\"Value of B:\", data_class_instance.get(\"B\"))"
 ]
},
{
 "cell_type": "code",
 "execution_count": 51,
 "metadata": {},
 "outputs": [
  {
   "data": {
    "text/plain": [
     "data_class({'A': 1, 'B': 'Foo'})"
    ]
   },
   "execution_count": 51,
   "metadata": {},
   "output_type": "execute_result"
  }
```

```
 ],
 "source": [
  "with open('file.py',\"w\") as f: \n",
  "    f.write(repr(data_class_instance))\n",
  "\n",
  "with open('file.py', 'r') as f: \n",
  "    data_class_instance_reloaded = eval(f.read())\n",
  "\n",
  "data_class_instance_reloaded"
 ]
},
{
 "cell_type": "code",
 "execution_count": 52,
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "data_class({'A': 1, 'B': 'Foo'})"
   ]
  }
 ],
 "source": [
  "!cat file.py"
 ]
},
{
 "cell_type": "code",
 "execution_count": 53,
 "metadata": {},
 "outputs": [],
 "source": [
  "data_class_instance_reloaded.add(\"C\",2)"
 ]
},
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "### pickle\n",
  "\n",
  "Pickle allows me to store the object:"
 ]
},
{
 "cell_type": "code",
 "execution_count": 54,
 "metadata": {},
 "outputs": [],
 "source": [
  "with open('file.pickle',\"wb\") as f: \n",
  "    pickle.dump(data_class_instance,f)"
```

```
 ]
},
{
 "cell_type": "code",
 "execution_count": 55,
 "metadata": {},
 "outputs": [
  {
   "data": {
    "text/plain": [
     "data_class({'A': 1, 'B': 'Foo'})"
    ]
   },
   "execution_count": 55,
   "metadata": {},
   "output_type": "execute_result"
  }
 ],
 "source": [
  "with open('file.pickle', 'rb') as f: \n",
  "    data_class_instance_reloaded = pickle.load(f)\n",
  "\n",
  "data_class_instance_reloaded"
 ]
},
{
 "cell_type": "code",
 "execution_count": 56,
 "metadata": {},
 "outputs": [
  {
   "data": {
    "text/plain": [
     "__main__.data_class"
    ]
   },
   "execution_count": 56,
   "metadata": {},
   "output_type": "execute_result"
  }
 ],
 "source": [
  "type(data_class_instance_reloaded)"
 ]
},
{
 "cell_type": "code",
 "execution_count": 57,
 "metadata": {},
 "outputs": [],
 "source": [
  "data_class_instance_reloaded.add(\"C\",2)"
 ]
},
```

```
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "## Storing Multiple Objects into Pickle\n",
  "\n",
  "Use a dictionary."
 ]
},
{
 "cell_type": "code",
 "execution_count": 58,
 "metadata": {},
 "outputs": [],
 "source": [
  "data_class_instance_2 = data_class()\n",
  "data_class_instance_2.add(\"C\",2)\n",
  "data_class_instance_2.add(\"D\",\"Bar\")"
 ]
},
{
 "cell_type": "code",
 "execution_count": 59,
 "metadata": {},
 "outputs": [],
 "source": [
  "with open('file.pickle',\"wb\") as f: \n",
  "    pickle.dump({\"my_class\":data_class_instance,\n",
  "                 \"my_class_2\":data_class_instance_2},\n",
  "                f)"
 ]
},
{
 "cell_type": "code",
 "execution_count": 60,
 "metadata": {},
 "outputs": [],
 "source": [
  "with open('file.pickle', 'rb') as f: \n",
  "    loaded_data = pickle.load(f)\n",
  "\n",
  "data_class_instance_reloaded = loaded_data[\"my_class\"]\n",
  "data_class_instance_reloaded_2 = loaded_data[\"my_class_2\"]"
 ]
},
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "## Pickling Data"
 ]
},
{
 "cell_type": "code",
```

```
   "execution_count": 61,
   "metadata": {},
   "outputs": [],
   "source": [
    "import numpy as np\n",
    "M = np.random.random((1000,1000))"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 62,
   "metadata": {},
   "outputs": [],
   "source": [
    "with open('M.pickle',\"wb\") as f: \n",
    "    pickle.dump(M, f)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 63,
   "metadata": {},
   "outputs": [],
   "source": [
    "np.save(\"M.npy\",M)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 64,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "total 48040\r\n",
      "-rwxr--r--@ 1 afarbin  staff    38K Feb 17 13:42 \u001b[31mLecture.10.a.ipynb\u001b[m\u001b[m\r\n",
      "-rwxr-xr-x@ 1 afarbin  staff   132K Feb 17 11:36 \u001b[31mLecture.10.b.ipynb\u001b[m\u001b[m\r\n",
      "-rw-r--r--@ 1 afarbin  staff   7.6M Feb 17 13:44 M.npy\r\n",
      "-rw-r--r--@ 1 afarbin  staff   7.6M Feb 17 13:44 M.pickle\r\n",
      "-rw-r--r--@ 1 afarbin  staff   7.6M Feb  7 10:37 M_list.pickle\r\n",
      "-rwxr--r--@ 1 afarbin  staff   2.4K Feb  7 10:37 \u001b[31mScores.csv\u001b[m\u001b[m\r\n",
      "-rw-r--r--@ 1 afarbin  staff    20B Feb 17 13:32 file.json\r\n",
      "-rw-r--r--@ 1 afarbin  staff   132B Feb 17 13:43 file.pickle\r\n",
      "-rw-r--r--@ 1 afarbin  staff    32B Feb 17 13:42 file.py\r\n",
      "-rw-r--r--@ 1 afarbin  staff    12B Feb 17 13:30 file.yaml\r\n"
     ]
    }
   ],
```

```
   "source": [
    "!ls -lh"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 65,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "total 48040\r\n",
      "-rwxr--r--@ 1 afarbin  staff    40360 Feb 17 13:44
\u001b[31mLecture.10.a.ipynb\u001b[m\u001b[m\r\n",
      "-rwxr-xr-x@ 1 afarbin  staff   134721 Feb 17 11:36
\u001b[31mLecture.10.b.ipynb\u001b[m\u001b[m\r\n",
      "-rw-r--r--@ 1 afarbin  staff  8000128 Feb 17 13:44 M.npy\r\n",
      "-rw-r--r--@ 1 afarbin  staff  8000163 Feb 17 13:44 M.pickle\r\n",
      "-rw-r--r--@ 1 afarbin  staff  8000163 Feb  7 10:37
M_list.pickle\r\n",
      "-rwxr--r--@ 1 afarbin  staff     2428 Feb  7 10:37
\u001b[31mScores.csv\u001b[m\u001b[m\r\n",
      "-rw-r--r--@ 1 afarbin  staff       20 Feb 17 13:32 file.json\r\n",
      "-rw-r--r--@ 1 afarbin  staff      132 Feb 17 13:43
file.pickle\r\n",
      "-rw-r--r--@ 1 afarbin  staff       32 Feb 17 13:42 file.py\r\n",
      "-rw-r--r--@ 1 afarbin  staff       12 Feb 17 13:30 file.yaml\r\n"
     ]
    }
   ],
   "source": [
    "!ls -l"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 66,
   "metadata": {},
   "outputs": [],
   "source": [
    "M_list=M.tolist()"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 67,
   "metadata": {},
   "outputs": [],
   "source": [
    "with open('M_list.pickle',\"wb\") as f: \n",
    "    pickle.dump(M, f)"
   ]
```

```
  },
  {
   "cell_type": "code",
   "execution_count": 68,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "total 48040\r\n",
      "-rwxr--r--@ 1 afarbin  staff    39K Feb 17 13:44
\u001b[31mLecture.10.a.ipynb\u001b[m\u001b[m\r\n",
      "-rwxr-xr-x@ 1 afarbin  staff   132K Feb 17 11:36
\u001b[31mLecture.10.b.ipynb\u001b[m\u001b[m\r\n",
      "-rw-r--r--@ 1 afarbin  staff   7.6M Feb 17 13:44 M.npy\r\n",
      "-rw-r--r--@ 1 afarbin  staff   7.6M Feb 17 13:44 M.pickle\r\n",
      "-rw-r--r--@ 1 afarbin  staff   7.6M Feb 17 13:45
M_list.pickle\r\n",
      "-rwxr--r--@ 1 afarbin  staff   2.4K Feb  7 10:37
\u001b[31mScores.csv\u001b[m\u001b[m\r\n",
      "-rw-r--r--@ 1 afarbin  staff    20B Feb 17 13:32 file.json\r\n",
      "-rw-r--r--@ 1 afarbin  staff   132B Feb 17 13:43 file.pickle\r\n",
      "-rw-r--r--@ 1 afarbin  staff    32B Feb 17 13:42 file.py\r\n",
      "-rw-r--r--@ 1 afarbin  staff    12B Feb 17 13:30 file.yaml\r\n"
     ]
    },
    {
     "name": "stderr",
     "output_type": "stream",
     "text": [
      "Bad pipe message: %s
[b'W\\x15\\x91\\x87\\x1f\\x99\\x8a\\xf0\\xe2,\\x7f\\xbb\\xd1\\xc5n\\x02\\
x9bu']\n",
      "Bad pipe message: %s [b'\\xa7\\x18\\xa1-
\\xfc\\x82qA\\xad7\\x87z\\xf2\\xf7\\xf1\\xac;\\xb6\\x00\\x01|\\x00\\x00\\
x00\\x01\\x00\\x02\\x00\\x03\\x00\\x04\\x00\\x05\\x00\\x06\\x00\\x07\\x00
\\x08\\x00\\t\\x00\\n\\x00\\x0b\\x00\\x0c\\x00\\r\\x00\\x0e\\x00\\x0f\\x0
0\\x10\\x00\\x11\\x00\\x12\\x00\\x13\\x00\\x14\\x00\\x15\\x00\\x16\\x00\\
x17\\x00\\x18\\x00\\x19\\x00\\x1a\\x00\\x1b\\x00/\\x000\\x001\\x002\\x003
\\x004\\x005\\x006\\x007\\x008\\x009\\x00:\\x00;\\x00<\\x00=\\x00>\\x00?\
\x00@\\x00A\\x00B\\x00C\\x00D\\x00E\\x00F\\x00g\\x00h\\x00i\\x00j\\x00k\\
x00l\\x00m\\x00\\x84\\x00\\x85\\x00\\x86\\x00\\x87\\x00\\x88\\x00\\x89\\x
00\\x96\\x00\\x97\\x00\\x98\\x00\\x99\\x00\\x9a\\x00\\x9b\\x00\\x9c\\x00\
\x9d\\x00\\x9e\\x00\\x9f\\x00\\xa0\\x00\\xa1\\x00\\xa2\\x00\\xa3\\x00\\xa
4\\x00\\xa5\\x00\\xa6\\x00\\xa7\\x00\\xba\\x00\\xbb\\x00\\xbc\\x00\\xbd\\
x00\\xbe\\x00\\xbf\\x00\\xc0\\x00\\xc1\\x00\\xc2\\x00\\xc3\\x00\\xc4\\x00
\\xc5\\x13\\x01\\x13\\x02\\x13', b'\\x04\\x13',
b'\\x01\\xc0\\x02\\xc0']\n",
      "Bad pipe message: %s [b'\\x04\\xc0']\n",
      "Bad pipe message: %s [b'\\x06\\xc0\\x07\\xc0']\n",
      "Bad pipe message: %s
[b'\\xdf\\xfd\\x8c\\xad\\xf2\\xee\\xdf6\\xf6\"r\\x06\\xa9\\x0b\\xf5\\x96\
\xe9K\\x00\\x01|\\x00\\x00\\x00\\x01\\x00\\x02\\x00\\x03\\x00\\x04\\x00\\
```

```
x05\\x00\\x06\\x00\\x07\\x00\\x08\\x00\\t\\x00\\n\\x00\\x0b\\x00\\x0c\\x0
0\\r\\x00\\x0e\\x00\\x0f\\x00\\x10\\x00\\x11\\x00\\x12\\x00\\x13\\x00\\x1
4\\x00\\x15\\x00\\x16\\x00\\x17\\x00\\x18\\x00\\x19\\x00\\x1a\\x00\\x1b\\
x00/\\x000\\x001\\x002\\x003\\x004\\x005\\x006\\x007\\x008\\x009\\x00:\\x
00;\\x00<\\x00=\\x00>\\x00?\\x00@\\x00A\\x00B\\x00C\\x00D\\x00E\\x00F\\x0
0g\\x00h\\x00i\\x00j\\x00k\\x00l\\x00m\\x00\\x84\\x00\\x85\\x00\\x86\\x00
\\x87\\x00\\x88\\x00\\x89\\x00\\x96\\x00\\x97\\x00\\x98\\x00\\x99\\x00\\x
9a\\x00\\x9b\\x00\\x9c\\x00\\x9d\\x00\\x9e\\x00\\x9f']\n",
      "Bad pipe message: %s [b'\\t\\xc0\\n\\xc0\\x0b\\xc0\\x0c']\n",
      "Bad pipe message: %s [b'2TW\\x97.-
\\xc0\\x14\\xfbrs;\\xfb\\xa1\\xc2\\x85\\xf4Q\\x00\\x01|\\x00\\x00\\x00\\x
01\\x00\\x02\\x00\\x03\\x00\\x04\\x00\\x05\\x00\\x06\\x00\\x07\\x00\\x08\
\x00\\t\\x00\\n\\x00\\x0b\\x00',
b'\\r\\x00\\x0e\\x00\\x0f\\x00\\x10\\x00\\x11\\x00\\x12']\n",
      "Bad pipe message: %s
[b'\\xe1w\\xe7.\\xbf\\xe8\\x04\\xf5\\x99NY\\x11u\\xd9\\x84\\xfeQ\\\\\\x00
\\x01|\\x00\\x00\\x00\\x01\\x00\\x02\\x00\\x03\\x00\\x04\\x00\\x05\\x00\\
x06\\x00\\x07\\x00\\x08\\x00\\t\\x00\\n\\x00\\x0b\\x00\\x0c\\x00\\r\\x00\
\x0e\\x00\\x0f\\x00\\x10\\x00\\x11\\x00\\x12\\x00\\x13\\x00\\x14\\x00\\x1
5\\x00\\x16\\x00\\x17\\x00\\x18\\x00\\x19\\x00\\x1a\\x00',
b'/\\x000\\x001\\x002\\x003\\x004\\x005\\x006\\x007\\x008\\x009\\x00:\\x0
0;\\x00']\n",
      "Bad pipe message: %s [b'-
\\xbe\\x11\\xd7\\xc2\\x91\\x1c\\x8f\\xb9\\xba\\xec\\xae\\x16i\\x8bt\\xcb\
\xb9\\x00\\x01|\\x00\\x00\\x00', b'']\n",
      "Bad pipe message: %s
[b'\\xb8\\x9dg\\xd7\\xf3\\xc6\\xfb\\xf5\\xc0\\xd5\\xcd\\xb4\\xa4\\xcc\\xf
d.TA\\x00\\x01|\\x00\\x00\\x00\\x01\\x00\\x02\\x00\\x03\\x00\\x04\\x00\\x
05\\x00\\x06\\x00\\x07\\x00\\x08\\x00\\t\\x00\\n\\x00\\x0b\\x00\\x0c\\x00
\\r\\x00\\x0e\\x00']\n",
      "Bad pipe message: %s
[b'=\\x00>\\x00?\\x00@\\x00A\\x00B\\x00C\\x00D\\x00E\\x00F\\x00g\\x00h\\x
00i\\x00j\\x00k\\x00l\\x00m\\x00\\x84\\x00\\x85\\x00\\x86\\x00\\x87\\x00\
\x88\\x00\\x89\\x00\\x96\\x00\\x97\\x00\\x98\\x00\\x99\\x00\\x9a\\x00\\x9
b\\x00\\x9c']\n",
      "Bad pipe message: %s [b'\\x03']\n",
      "Bad pipe message: %s
[b'\\x10\\x00\\x11\\x00\\x12\\x00\\x13\\x00\\x14\\x00\\x15\\x00\\x16\\x00
']\n",
      "Bad pipe message: %s
[b'\\x18\\x00\\x19\\x00\\x1a\\x00\\x1b\\x00/\\x000\\x001\\x002\\x003\\x00
4\\x005\\x00']\n",
      "Bad pipe message: %s
[b'7\\x008\\x009\\x00:\\x00;\\x00<\\x00=\\x00>\\x00?\\x00@\\x00A\\x00B\\x
00C\\x00D\\x00E\\x00F\\x00g\\x00h\\x00i\\x00j\\x00k\\x00l\\x00m\\x00\\x84
\\x00\\x85\\x00\\x86\\x00\\x87']\n"
     ]
    }
   ],
   "source": [
    "!ls -lh"
   ]
  },
  {
```

```
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "!ls -l"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "!rm *.pickle *.yaml *.json file.py"
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3 (ipykernel)",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.13.12"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 4
}
```